

3.28 mmbucket 多次元均等化バケット分割

`f=`で指定した複数の数値項目を次元とした件数均等化バケット分割を行う。例えば、`f=a,b,c`そして`n=5`と指定すると、`mbucket` コマンドと同様に、項目 `a,b,c` それぞれを 5 つの区間に分割するが、`mmbucket` では、項目 `a,b,c` の 3 次元空間における各バケット (バケット数は $5^3 = 125$ 個になる) に属する件数ができるだけ均等になるような区間を計算する

書式

```
mmbucket f= n= [F=] [k=] [0=] [-ms] [-r] [i=] [o=] [bufcount=] [-assert_diffSize] [-assert_nullkey] [-assert_nullin] [-assert_nullout] [-nfn] [-nfno] [-x] [-q] [tmpPath=] [--help] [--help1] [--version]
```

パラメータ

- f=** ここで指定した項目 (複数項目指定可) の値を分割する。
複数指定すれば、その数の次元に基づく均等化バケット分割を行う。
1 項目のみ指定すれば `mbucket` と同じ結果になる。
`-x, -nfn` オプション使用時は、項目番号 (0~) で指定可能。
- n=** `f=`で指定した項目数と同じ個数分指定する。
ただし 1 つの数字を指定した場合、`f=`で指定した全ての項目に、同じ分割数が適用される。
- F=** 出力形式を指定する。【デフォルト値:1】
バケットの名前を出力形式。
0:バケット番号のみを表示する。
1:バケットの範囲のみを表示する。
2:バケット番号とバケット範囲の両方を表示する。
- k=** バケット分割を行う単位となる項目名リスト (複数項目指定可) を指定する。
- 0=** `f=`パラメータで指定した各項目の各バケットの数値範囲を出力するファイル名を指定する。
- ms** 各項目を順次バケット分割していく時の開始項目を変えることで複数回のバケット分割をトライし、よりよい解を求める。詳細は、以下の「アルゴリズムの概要」を参照のこと。
- r** バケット番号を逆順に出力する。

利用例

例 1: 基本例

`x, y` 項目の件数ができるだけ多次元均等になるように 2 分割する。その際、各バケットの数値範囲を `rng.csv` という名前のファイルに出力する。

```
$ more dat1.csv
id,x,y
A,2,7
B,6,7
C,5,6
D,7,5
E,6,4
F,1,3
G,3,3
H,4,2
I,7,2
J,2,1
$ mmbucket f=x:xb,y:yb n=2,2 0=rng.csv i=dat1.csv o=rsl1.csv
calculating on dimension ... #0 #1 done. VAR=30 updated!
```

```

calculating on dimension ... #0 #1 done. VAR=28 updated!
calculating on dimension ... #0 #1 done. VAR=28
#END# kgmbucket 0=rng.csv f=x:xb,y:yb i=dat1.csv n=2,2 o=rsl1.csv
$ more rsl1.csv
id,x,y,xb,yb
A,2,7,1,2
B,6,7,2,2
C,5,6,2,2
D,7,5,2,2
E,6,4,2,1
F,1,3,1,1
G,3,3,1,1
H,4,2,2,1
I,7,2,2,1
J,2,1,1,1
$ more rng.csv
fieldName,bucketNo,rangeFrom,rangeTo
x,1,,3.5
x,2,3.5,
y,1,,4.5
y,2,4.5,

```

例 2: 出力形式

id 項目を単位に件数ができるだけ多次元均等になるように x,y 項目を 2 分割する。出力形式はバケット番号とバケット範囲の両方を表示する。

```

$ more dat2.csv
id,x,y
A,2,7
A,6,7
A,5,6
B,7,5
B,6,4
B,1,3
C,3,3
C,4,2
C,7,2
C,2,1
$ mmbucket k=id f=x:xb,y:yb n=2,2 F=2 i=dat2.csv o=rsl2.csv
calculating on dimension ... #0 #1 done. VAR=3 updated!
calculating on dimension ... #0 #1 done. VAR=3
calculating on dimension ... #0 #1 done. VAR=3 updated!
calculating on dimension ... #0 #1 done. VAR=3
calculating on dimension ... #0 #1 done. VAR=6 updated!
calculating on dimension ... #0 #1 done. VAR=6
#END# kgmbucket F=2 f=x:xb,y:yb i=dat2.csv k=id n=2,2 o=rsl2.csv
$ more rsl2.csv
id%0,x,y,xb,yb
A,2,7,1:_3.5,2:6.5_
A,6,7,2:3.5_,2:6.5_
A,5,6,2:3.5_,1:_6.5
B,7,5,2:3.5_,2:4.5_
B,6,4,2:3.5_,1:_4.5
B,1,3,1:_3.5,1:_4.5
C,3,3,1:_3.5,2:1.5_
C,4,2,2:3.5_,2:1.5_
C,7,2,2:3.5_,2:1.5_
C,2,1,1:_3.5,1:_1.5

```

アルゴリズムの概要^{*2}

2つの数値項目からなるデータセット D があるものとし、2つの項目名を A, B とする。 A および B の数値範囲をそれぞれ K 個、 L 個に分割するものとする。この分割により、データセットは $K \cdot L$ 個に分割される。mmbucket は分割された各領域 (バケット) に入るデータの個数が一樣になるような分割方法を求める。一様性の評価基準としては分散を用いるものとする。分散値が最小のバケット分割を求めるものであるが、ここでは、以下に述べるようなヒューリスティックスに基づいており、最適性は保証していない。アルゴリズムは以下のように記述される。

1. 項目 A について件数均等化 1 次元バケット分割をおこなう。
2. 1. で得られた A の分割を固定して、 B の数値範囲のバケット分割をおこなう。このときの評価基準は 2 次元バケット分割の分散値である。方法は、件数均等化 1 次元バケット分割と同様に動的計画法を用いている。
3. 次にステップ 2 で得られた B の分割を固定して、 A の数値範囲のバケット分割をおこなう。このときの評価基準はステップ 2 と同様に、2 次元バケット分割の分散値である。
4. 以上のステップ 2,3 を分散値の改良が生じなくなるまで繰り返す。
5. 最終的に得られた分割を出力する。

実装においては、ステップ 1 において、 A と B の役割を入れ替えた場合も走らせて、解を求め、両者で得られた 2 つの解のうち優れた方を出力するようにしている。また、3 次元以上の場合の均等化バケット分割も同様のアルゴリズムに基づいている。

mbucket と mmbucket の比較

多次元バケット分割がどのように分割を行うかを示すために、1 次元バケット分割との比較を通して以下で説明する。表 3.5 には、二つの数値属性 x, y を持つ、id が A から J までの 10 件データが示されている。このデータについて、属性 x, y をそれぞれ 2 分割し計 4 つのバケットに分割することを考えよう。属性 x と y のそれぞれを 1 次元バケット分割することで得られた結果が $x1, y1$ 項目として示されており、その値を用いて 4 つのバケットに分割した様子が図に示されている。そして多次元バケット分割のアルゴリズムを用いて得られた結果が $x2, y2$ 項目として示されており、その分割の様子が図に示されている。

分散値 (各バケットに属するデータ件数の二乗和：詳しくは mbucket の定式化を参照) は、1 次元バケット分割においては、 $Var'_a = 1^2 + 4^2 + 4^2 + 1^2 = 34$ となり、2 次元バケット分割では $Var'_b = 1^2 + 3^2 + 3^2 + 3^2 = 28$ となる。1 次元バケット分割においては、 x, y それぞれ独立で考えた場合には、分散を最小化する最適解となっている (すなわち 5 件ずつに分割している) が、2 次元の分割として評価すると、2 次元バケット分割に比べて劣った分割となる。このように多次元バケット分割を使うことで 1 次元バケット分割の組み合わせよりも優れた分割が可能となる。しかしながら、次項で示すように、多次元分割はデータの内容によっては多大な計算時間が必要となるケースがあることに注意する。

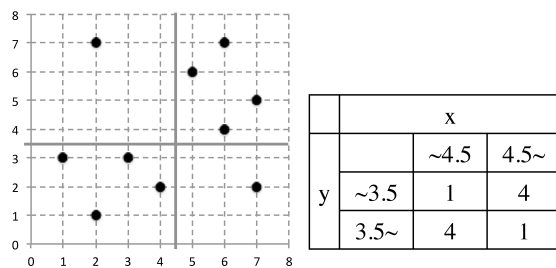


図 3.1 1 次元分割 (mbucket) $\times$ 2 による分割

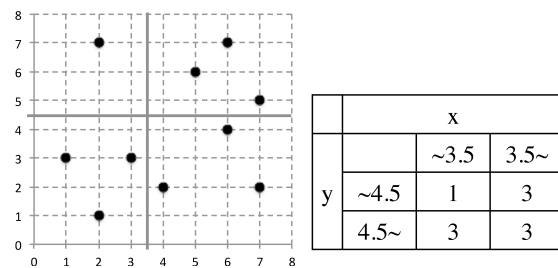


図 3.2 2 次元分割 (mmbucket) による分割

^{*2} このアルゴリズムは加藤直樹教授 (京都大学工学研究科) による。

表 3.5 サンプルデータと mbucket, mmbucket による出力結果

データ			mbucket		mmbucket	
id	x	y	x1	y1	x2	y2
A	2	7	1	2	1	2
B	6	7	2	2	2	2
C	5	6	2	2	2	2
D	7	5	2	2	2	2
E	6	4	2	2	2	1
F	1	3	1	1	1	1
G	3	3	1	1	1	1
H	4	2	1	1	2	1
I	7	2	2	1	2	1
J	2	1	1	1	1	1

多様な項目での精度実験

mbucket と mmbucket の精度を実データを用いて比較してみる。利用するデータは、エルニーニョ現象の解明を目的とした **TAO プロジェクト** で収集されているデータで、赤道付近に設置されたセンサーが収集した海洋データおよび気象データである。表 3.8 にデータの一部を示す。

表 3.6 精度実験用データセット

id	year	month	day	date	latitude	longitude	zonwinds	merwinds	humidity	air_temp.	sstemp
4060	93	5	9	930509	-0.02	-109.96	-2.1	2.1	81.2	26.8	27.02
4061	93	5	10	930510	-0.02	-109.96	-3.4	1.4	84.2	26.95	26.91
4062	93	5	11	930511	-0.02	-109.96	-3.8	2.2	84.9	26.98	26.78
4063	93	5	12	930512	-0.02	-109.96	-3	1.5	86.9	26.93	26.74
4064	93	5	13	930513	-0.02	-109.96	-4.5	1.9	87.6	27.01	26.82
4065	93	5	14	930514	-0.02	-109.96	-5	1.3	85.6	26.96	26.68

latitude:緯度, longitude:経度, zonwinds:東西風速, merwinds:南北風速, humidity:湿度, air_temp.:気温, sstemp:海面温度

このデータの数値属性項目 (緯度から海面温度までの 7 項目) を使いバケット分割の精度実験を行った。レコード行数は 93,935 行である。各項目の統計は表 3 に示す通りである。バケット分割においては「値の種類数」が計算時間に大きく影響を及ぼす。

表 3.7 数値データの各種統計

計量	緯度	経度	東西風速	南北風速	湿度	気温	海面温度
値の種類数	482	924	228	206	385	1104	1201
算術平均	0.305	-70.8	-3.35	-0.046	81.3	27.1	27.9
標準偏差	4.77	128.7	3.42	3.021	5.28	1.674	1.87
最小値	-8.33	-180	-10.7	-10.6	52.1	17.5	18.2
中央値	0.01	-125	-4.1	-0.1	81.3	27.5	28.4
最大値	9.05	170.0	14.3	13	99.9	31.5	31.0

7つの属性から2項目の組み合わせ全 21 通りについて 1次元バケット分割 (mbucket) に基づく方法と 2次元バケット分割 (mmbucket) に基づく方法の分散値 (バケットに属する件数の二乗和) の比較が表 3.8 に示されている。例えば一行目に示された結果は、気温×湿度のバケット分割の比較で、mbucket での分散値が 408274409、mmbucket での分散値が 406438211 であり、その比 (mmbucket/mbucket) が 0.996 であることが示されている。続いて、分割数を 10 × 10、15 × 15、20 × 20 としたときの比が示されている。結果は属性の組み合わせによって異なり、「湿度×東西風速」のように全く改善の見られない属性の組み合わせもあれば、「緯度×経度 (15 × 15)」のように 19%(1-0.81) の精度向上を示す属性もみられる。

表 3.8 mbucket と mmbucket による 2 次元分割精度の比較

項目 1	項目 2	分散値 (5 × 5 分割)		分散値の比 (mmbucket/mbucket)			
		mbucket	mmbucket	5 × 5	10 × 10	15 × 15	20 × 20
気温	湿度	408274409	406438211	0.996	0.987	0.988	0.981
	緯度	374125463	371258775	0.992	0.988	0.982	0.964
	経度	490727955	454663065	0.927	0.949	0.946	0.929
	南北風速	396436813	394724219	0.996	0.991	0.993	0.989
	海面温度	816199131	747215787	0.915	0.897	0.883	0.862
	東西風速	382069455	381225143	0.998	0.998	0.997	0.996
湿度	緯度	368492959	367941709	0.999	0.994	0.994	0.990
	経度	372116309	370591351	0.996	0.991	0.993	0.986
	南北風速	355658757	355658757	1.000	1.000	1.000	1.000
	海面温度	380382203	379546293	0.998	0.991	0.991	0.983
	東西風速	357729819	357697283	1.000	1.000	1.000	1.000
緯度	経度	365459223	361754113	0.990	0.923	0.812	0.816
	南北風速	371478669	370970251	0.999	0.988	0.985	0.982
	海面温度	392810425	389589403	0.992	0.987	0.979	0.952
	東西風速	364521077	364406663	1.000	0.999	0.991	0.992
経度	南北風速	414154185	408431667	0.986	0.976	0.976	0.962
	海面温度	510979465	463576537	0.907	0.945	0.939	0.934
	東西風速	400021527	392710641	0.982	0.983	0.982	0.973
南北風速	海面温度	393233841	392432943	0.998	0.995	0.994	0.993
	東西風速	359290669	359074015	0.999	1.000	1.000	0.997
海面温度	東西風速	442877539	438326669	0.990	0.988	0.984	0.986

速度比較

次に分割数による速度の違いみるための実験を行った。前節と同様のデータを用い、気温×海面温度および緯度×経度について、分割数を 5 から 40 まで 5 刻みで設定し、その実行時間を計測した。その結果が表 3.9 に示されている。mbucket の実行時間は分割数に関係なくほぼ一定である。一方で 2 次元分割については分割数が多くなるに従って実行時間も長くなっている。これはアルゴリズム内部で必要となる多次元直交数値範囲の選択において効率的なアルゴリズムを実装していないためである。これは改善可能であるが、次期バージョンでの課題としたい。

表 3.9 サンプルデータと mbucket, mmbucket による出力結果

分割数	気温×海面温度		緯度×経度	
	mbucket	mmbucket	mbucket	mmbucket
5	0.221	2.21	0.216	0.67
10	0.227	3.90	0.216	1.67
15	0.233	10.4	0.230	3.28
20	0.231	26.1	0.228	7.13
25	0.232	32.9	0.237	13.3
30	0.236	46.7	0.236	11.4
35	0.237	62.3	0.240	15.2
40	0.237	80.1	0.237	25.8

関連コマンド

mbucket : 複数項目指定しても、それぞれの項目で 1 次元バケット分割を実行する。